

AI-Powered E-Commerce Analytics Platform

Secure natural-language analytics, interactive visualization, advanced anomaly detection, and decision-ready reporting

Programme / Module	MSc Summer 2026 — Data Visualization
Lead author	S. M. Monowar Kayser
Student ID	[INSERT STUDENT ID]
Team members	[INSERT TEAM MEMBER NAMES, IF APPLICABLE]
Repository	github.com/Monokayser/ai-ecommerce-analytics
Submission date	[INSERT FINAL SUBMISSION DATE]

Evidence status

This report distinguishes implemented functionality from verified results. Performance and analytical figures use the deterministic 2,000-row synthetic development dataset (seed 42). The official 5,000+ row dataset and live-provider benchmark remain final verification gates.

RESEARCH SYNOPSIS

Abstract and Contents

Abstract

This capstone presents a production-style analytics platform that converts heterogeneous e-commerce data into verified analytical evidence. The system accepts CSV, JSON, and Parquet files; preserves raw and cleaned views; profiles schema and quality; executes parameterized DuckDB queries; and offers six responsive Streamlit workspaces. Natural-language questions are converted into typed SQL or restricted pandas plans by an adaptive provider-neutral AI layer. Every generated plan is validated with Pydantic and an abstract-syntax-tree allowlist before execution; model text never executes directly. The platform also provides eight visualization families, IQR and Isolation Forest anomaly detection, two-subset comparison, conversation memory, and Word/PDF/PNG/SVG export. Evaluation on the deterministic 2,000-row synthetic dataset produced 72 passing tests, 84% statement coverage, zero Bandit findings, no known dependency vulnerabilities, and warmed DuckDB median query latencies of 14.85–15.80 ms across three representative aggregations. Live LLM benchmark rows remain explicitly not run because no provider key was configured; therefore no model accuracy is claimed. The result is a reproducible, secure, and extensible decision-support environment aligned with the capstone rubric while clearly documenting its evidence limits.

Keywords: e-commerce analytics; natural-language querying; DuckDB; Streamlit; secure AI; anomaly detection; data visualization

Contents

Section	Title	Page
1	Introduction and problem definition	3
2	Requirements and development methodology	4
3	System architecture	5
4	Data and database design	6
5	Preprocessing and quality management	7
6	User-interface design	8
7	Visualization and interaction design	9
8	AI model and natural-language pipeline	10
9	Security and execution workflow	11
10	Anomaly detection	12
11	Comparative analysis	13
12	Implementation and DevOps	14
13	Testing and evaluation	15
14	Performance analysis	16
15	AI benchmark and reliability	17
16	Limitations, ethics, and responsible AI	18
17	Future work and conclusion	19
References and disclosure		20

CONTEXT

1 Introduction and Problem Definition

1.1 Problem statement

Retail datasets are often fragmented across spreadsheet and export formats, contain inconsistent headers and data types, and require specialist knowledge before they can answer ordinary business questions. Traditional dashboards provide predetermined views but do not readily translate ad hoc natural-language questions into reproducible calculations. Conversely, unconstrained generative AI can produce plausible but unsafe or unsupported answers. The central problem is therefore not simply query generation: it is the controlled transformation of a user question into validated, executable, and explainable analytical evidence.

1.2 Aim and objectives

The aim was to build a self-contained e-commerce intelligence workspace that remains useful without an API key and becomes more linguistically flexible when a hosted or local model is available. The platform was designed against the official capstone brief [1] and the stricter production requirements supplied for this implementation.

- Ingest and validate realistic CSV, JSON, and Parquet datasets while protecting the application from unsafe files and ambiguous schemas.
- Support reproducible direct analytics and natural-language querying through a provider-neutral, schema-constrained AI pipeline.
- Present results through responsive KPIs and at least eight interactive chart families with consistent accessible styling.
- Implement two advanced analytical features: anomaly detection and comparative analysis.
- Export decision-ready Word, PDF, PNG, and SVG artifacts with filters, evidence, limitations, and AI-validation disclaimers.
- Measure correctness, security, latency, and maintainability without fabricating live-provider results.

1.3 Scope and deliverables

The delivered repository contains a layered Streamlit application, typed contracts, synthetic development data, deployment and security configuration, automated tests, benchmark specifications, Docker support, documentation, this maximum-20-page report, and a 15-minute presentation. External publication and an official-dataset benchmark are deliberately treated as deployment gates rather than inferred achievements.

Research question

How can a natural-language e-commerce analytics interface improve accessibility while retaining deterministic computation, explicit security controls, reproducibility, and evidence-based communication?

PLANNING

2 Requirements and Development Methodology

2.1 Requirements traceability

Requirement	Implemented response	Verification
Data ingestion	CSV/JSON/Parquet; 200 MB limit; sanitized names; lazy path for >1M rows	Loader and hardening tests
Dashboard	Six sections, persistent filters, KPIs, responsive theme, empty states	Streamlit AppTest and browser QA
AI assistant	Offline planner plus Gemini/OpenAI/Ollama adapters; structured plans	Provider, offline, sandbox tests
Secure execution	Single SELECT/WITH, dataset-only SQL, restricted pandas AST, time/row limits	Adversarial security tests
Visualization	Eight required chart families plus typed automatic selection	Constructor smoke tests
Advanced analysis	IQR + Isolation Forest; two-subset KPI comparison	Calculation and edge-case tests
Export	Word/PDF reports and Plotly PNG/SVG	Reopen/signature smoke tests
Delivery	README, Docker, CI, security policy, Git history, GitHub release	Repository audit

Table 1. Capstone requirements mapped to implementation and verification evidence.

2.2 Iterative development methodology

An incremental engineering method was used. Each vertical slice began with typed contracts and deterministic business logic, then added the Streamlit presentation layer, automated tests, and operational documentation. Security controls were implemented at the same boundary as query generation rather than added after interface completion. This reduced coupling and allowed the offline planner, hosted providers, and direct analytics to share one execution and visualization path.

Phase	Primary activities	Exit evidence
Discover	Rubric decomposition; risks; schema and interface contracts	Traceability matrix
Build	Data layer, query engine, UI, AI adapters, charts, exports	Runnable vertical slices
Harden	AST allowlists, upload controls, retries, error handling	Negative security tests
Evaluate	Unit/AppTest/export/benchmark/performance checks	Measured test artefacts
Package	Docs, Docker, GitHub release, report, presentation	Reproducible hand-off

Table 2. Development workflow and evidence gates.

LAYERED DESIGN

3 System Architecture

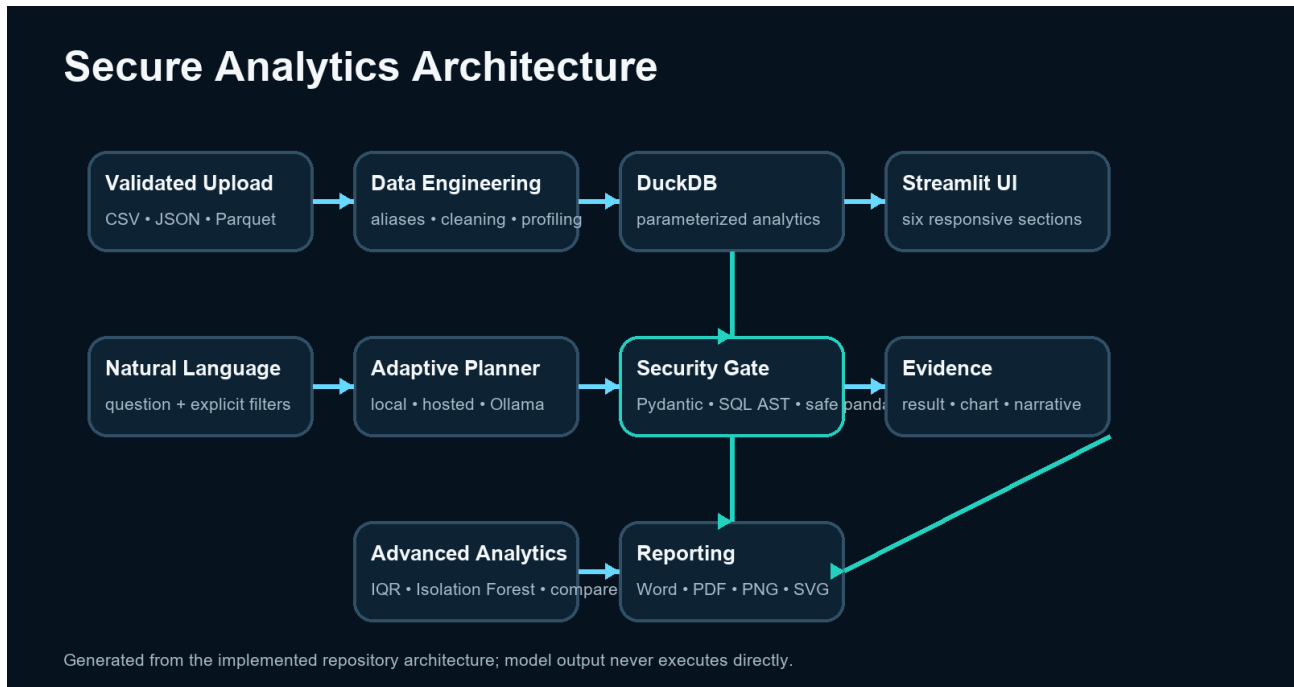


Figure 1. Implemented layered architecture and controlled natural-language execution path.

3.1 Architectural responsibilities

The Streamlit shell initializes configuration, state, and routing only. UI-independent logic is grouped into data, LLM, visualization, advanced analytics, reporting, and utility modules. Pydantic contracts—such as DatasetBundle, GeneratedQuery, QueryResult, ChartSpec, AnomalyResult, ComparisonResult, and ReportPayload—form stable boundaries between layers. DuckDB provides in-process analytical SQL and direct pandas/Arrow interoperability [3], while Streamlit session state preserves active filters, page selection, dataset context, and five compact conversation records [2].

3.2 End-to-end flow

A validated upload is canonicalized, cleaned, profiled, and registered as the single logical table dataset. Direct dashboard calculations use parameterized query specifications. Natural-language questions pass through a planner, typed-response validation, SQL or pandas security validation, bounded execution, evidence summarization, chart selection, and reporting. Explicit current filters override conversational context. Model output is advisory until the deterministic security and execution stages accept it.

Design principle

The architecture separates interpretation from computation: AI proposes a typed plan; application code validates, executes, summarizes, and visualizes it.

DATA ENGINEERING

4 Data and Database Design

4.1 Development dataset

The repository ships a deterministic synthetic e-commerce CSV generated with seed 42. It contains 2,000 rows and 14 source columns covering transactions, dates, merchandise hierarchy, customer segment, geography, measures, and fulfilment. It is visibly labelled demo data. The application reports official-demo readiness only when at least 5,000 rows are loaded, consistent with the assignment brief [1]. No claim is made that the development dataset represents a real retailer.

Field	Type / role	Field	Type / role
Order ID	text / identifier	Order Date	date / time
Ship Date	date / time	Product Category	text / dimension
Sub-Category	text / dimension	Customer Segment	text / dimension
Region	text / geography	Country	text / geography
City	text / geography	Sales	decimal / measure
Quantity	integer / measure	Discount	decimal / measure
Profit	decimal / measure	Ship Mode	text / dimension

Table 3. Source schema of the synthetic development dataset.

4.2 Logical database model

The application uses a single analytical relation named dataset rather than a transactional server database. Each query receives a fresh DuckDB connection; filters are bound as parameters; output is limited to 1,000 rows by default; and a 10-second interrupt timer bounds execution. The model is intentionally denormalized because the workload is read-heavy, aggregate-oriented, and scoped to one user-supplied analytical file. Raw and cleaned frames remain separately addressable so transformations are auditable and source values are not overwritten.

4.3 Canonical schema and aliases

A configurable alias registry maps common variants to canonical business fields. Exact canonical names take precedence. Ambiguous aliases are reported rather than silently renamed, preventing two source fields from being collapsed. Missing optional columns disable only dependent metrics and charts, maintaining graceful degradation instead of rejecting the entire dataset.

REVERSIBLE TRANSFORMATION

5 Preprocessing and Data-Quality Management

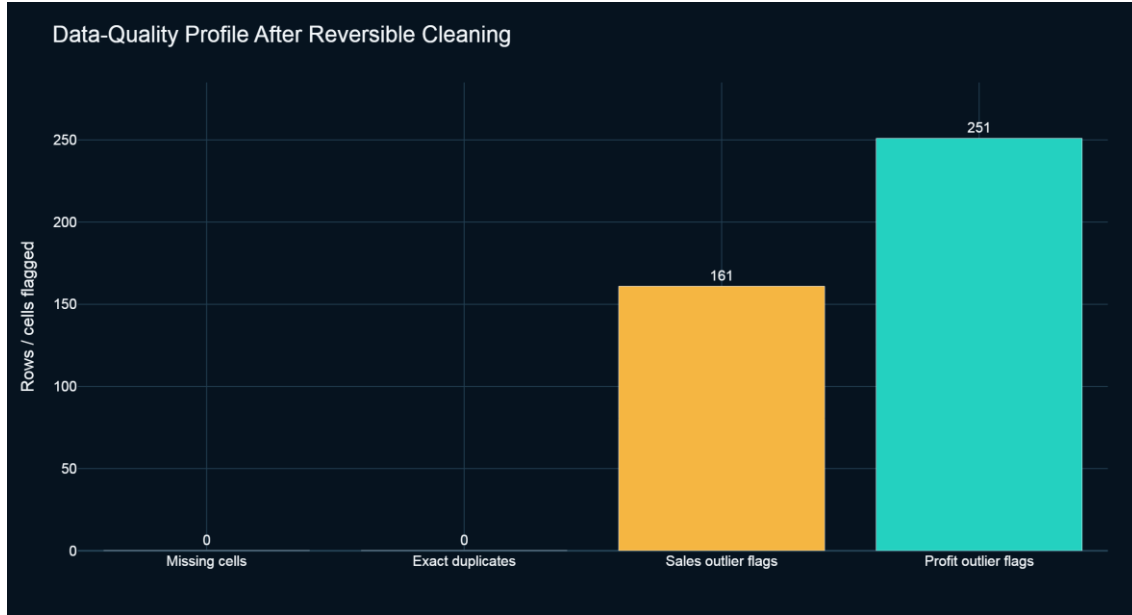


Figure 2. Measured quality indicators for the cleaned 2,000-row development dataset.

5.1 Cleaning pipeline

Default cleaning parses Order Date and Ship Date; safely converts Discount, Sales, Quantity, and Profit to numeric types; trims and normalizes categorical text; removes exact duplicate rows; and adds IQR-based outlier markers for numerical measures. pandas provides the tabular transformation layer [4]. Each CleaningAction records the affected field, operation, count, and explanatory detail. Outliers are not deleted or capped because unusual values may be legitimate business events and are required by the advanced analytics task.

Measured indicator	Result	Interpretation
Raw / cleaned rows	2,000 / 2,000	No source records removed
Source / cleaned columns	14 / 18	Four reversible outlier-marker fields added
Missing cells	0	No imputation required for the synthetic file
Exact duplicates	0	Duplicate rule executed; no matches
Sales IQR flags	161 (8.05%)	Marked only; requires business review
Profit IQR flags	251 (12.55%)	Marked only; not labelled fraud/error
Raw memory	0.3924 MB	Development dataset footprint

Table 4. Data-quality and preprocessing evidence.

5.2 Profiling and LLM-safe context

The profiler calculates data type, semantic role, nulls, uniqueness, descriptive statistics, bounded samples, and quality notes. Sample cells are truncated and delimited as untrusted quoted data before they enter a prompt. This provides enough context to ground field selection while preventing cell text from becoming instructions or exposing large portions of the dataset.

RESPONSIVE WORKSPACE

6 User-Interface and User-Experience Design

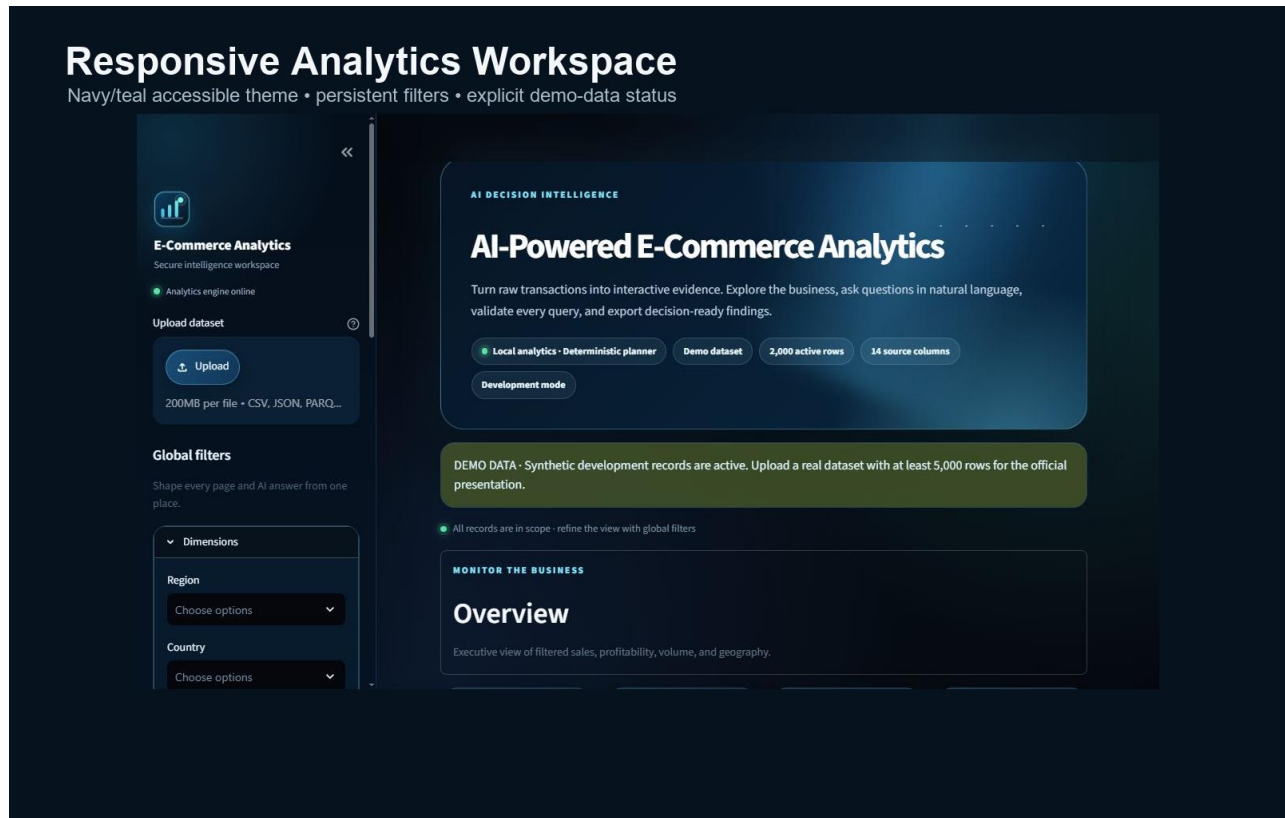


Figure 3. Overview workspace showing the accessible navy/teal theme and explicit demo-data status.

6.1 Information architecture

The interface is organized into Overview, Data Exploration, AI Assistant, Advanced Analytics, Data Quality & Performance, and Report Export. The sidebar centralizes upload controls and global dimensions so the active analytical scope is consistent across pages. Dataset, filters, page selection, and the five most recent AI interactions persist in session state. Reset controls clear filters and conversation state without mutating the source dataset.

6.2 Accessibility, responsiveness, and interaction

The visual system uses high-contrast navy surfaces, teal status cues, scalable system fonts, generous touch targets, descriptive labels, tooltips, visible focus states, reduced-motion support, and responsive card grids. KPI cards display concise formatting and preceding-period comparison when a suitable date range exists. Empty states explain missing columns or filters rather than showing broken charts. Subtle CSS transitions provide feedback without blocking interaction, while a lightweight perspective panel supplies a restrained 3D visual motif without introducing a heavy rendering framework.

6.3 AI interaction design

Users can choose Fast, Balanced, or Deep planning modes, see the current provider/fallback status, submit an example question, and review generated query, validation, execution timing, retry state, result summary, chart, and grounded answer. Hidden reasoning is never displayed. The conversation record stores only the question, typed plan, explicit filters, result summary, and final response.

ANALYTICAL COMMUNICATION

7 Visualization and Interaction Design

7.1 Visualization portfolio

Visualization	Primary analytical purpose	Availability / fallback
Time series	Sales/profit trend and period comparison	Requires date + numeric measure
Choropleth	Geographic distribution	Falls back to ranked geographic bars
Correlation heatmap	Pairwise numeric association	Requires multiple numeric fields
Histogram / box	Distribution and outlier context	Requires numeric field
Sunburst / treemap	Hierarchical contribution	Requires two dimensions + measure
Grouped / stacked bar	Category/subgroup comparison	Requires categorical dimensions
Scatter + trend	Relationship and directional pattern	Requires two numeric fields
Animated time chart	Change across time and categories	Requires date, category, measure

Table 5. Required visualization families and graceful-degradation rules.

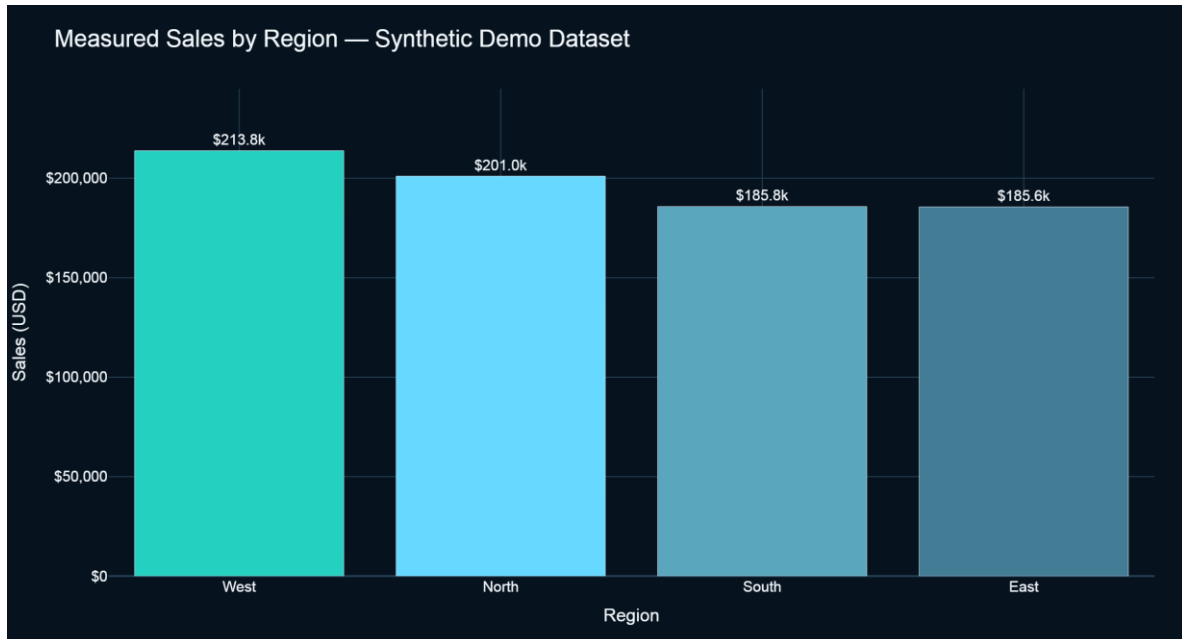


Figure 4. Measured regional sales summary rendered with the shared accessible chart theme.

7.2 Automatic chart selection and caption validation

The ChartSelector inspects result cardinality and typed columns. Dates prioritize lines, geographic dimensions prefer maps, two numerics prefer scatter plots, and categorical aggregations prefer bars or hierarchical views. Users may override the recommendation. Plotly supplies interactive hover, zoom, selection, animation, geographic, hierarchical, statistical, and 3D chart primitives [5]. Before display, captions are checked against computed result summaries so model-generated text cannot introduce unsupported values.

MODEL ENGINEERING

8 AI Model and Natural-Language Query Pipeline

8.1 Provider-neutral architecture

The LLMClient interface exposes `generate_query()` and `format_result()`. The preferred hosted path uses Gemini 3.6 Flash configured through the Google GenAI SDK, while OpenAI and Ollama remain optional adapters. Google's structured-output interface accepts JSON Schema and Pydantic-compatible definitions [7]. A deterministic local planner provides key-free operation for common e-commerce intents and is the automatic fallback when a provider is unavailable. The AI proposes a GeneratedQuery containing interpreted question, language, query, columns, filters, aggregation, chart, and concise rationale.

Mode	Planning behavior	Narrative behavior	Use case
Fast	Low-effort hosted plan or local plan	Deterministic computed summary	Live demos and common KPIs
Balanced	Medium-effort structured plan	Concise evidence-grounded AI text	Default general analysis
Deep	High-effort structured plan	High-effort grounded explanation	Ambiguous multi-step questions

Table 6. Adaptive AI modes and latency/quality trade-offs.

8.2 Prompt construction and response validation

Prompts include business context, canonical aliases, column types, bounded safe samples, active filters, query rules, and the exact output schema. Data values are placed in explicit untrusted-data delimiters and cannot override system instructions. Provider responses are parsed into Pydantic models; invalid, empty, truncated, or schema-incompatible output fails closed. Authentication, quota, timeout, and unavailable-model failures produce actionable messages and may trigger the deterministic fallback.

8.3 Training and evaluation status

No foundation model was trained or fine-tuned within this project. Consequently, train/validation splits, accuracy/loss curves, and confusion matrices are not applicable to the LLM component. The correct evaluation unit is the generated analytical plan: required fields, operation, filters, execution success, chart suitability, and grounded narrative. Ten benchmark specifications are stored for live-provider evaluation, but all rows remain `not_run` until a provider key is supplied.

TRUST BOUNDARY

9 Security and Controlled Execution Workflow

9.1 SQL security policy

DuckDB SQL is the primary generated language. sqlglot parses the query into an abstract syntax tree before execution. The validator permits exactly one SELECT or WITH statement, restricts relations to the registered dataset table, rejects DDL/DML, multiple statements, table functions, filesystem scans, attachment commands, network-capable extensions, and unsafe expressions, and then enforces the result-row ceiling. Filter values are parameterized rather than interpolated. A fresh connection and 10-second interrupt timer isolate each query.

9.2 Restricted pandas fallback

The pandas route uses a custom AST interpreter with allowlisted operations and attributes. Python `eval()` and `exec()` are never called. Imports, dunder access, dangerous built-ins, arbitrary attributes, filesystem and network operations, and mutation of the source frame are denied. This route exists for constrained operations that are inconvenient in SQL, not as a general code-execution facility.

Threat	Control	Expected outcome
Prompt injection in cells	Quote, truncate, delimit, schema-constrain	Cell text remains data
DDL/DML or multiple SQL	Parsed AST statement allowlist	Rejected before connection
Filesystem/network scan	Table/function denylist	No external resource access
Unsafe Python	Custom interpreter; no eval/exec/import	Rejected operation
Runaway query	Fresh connection; 10 s interrupt; row limit	Bounded execution
Hallucinated narrative	Narrate computed evidence only; caption checks	Unsupported claims blocked/disclosed
Secret exposure	Environment configuration; no raw-row logs	Keys and records excluded

Table 7. Threat model and implemented controls.

9.3 Recovery policy

A sanitized execution error may be returned to the selected provider for exactly one corrected-query attempt. The corrected plan is fully revalidated. Provider transport has one bounded retry for transient faults. Persistent failure produces a safe error or deterministic fallback rather than progressively relaxing policy.

TASK D3

10 Advanced Analytics I — Anomaly Detection

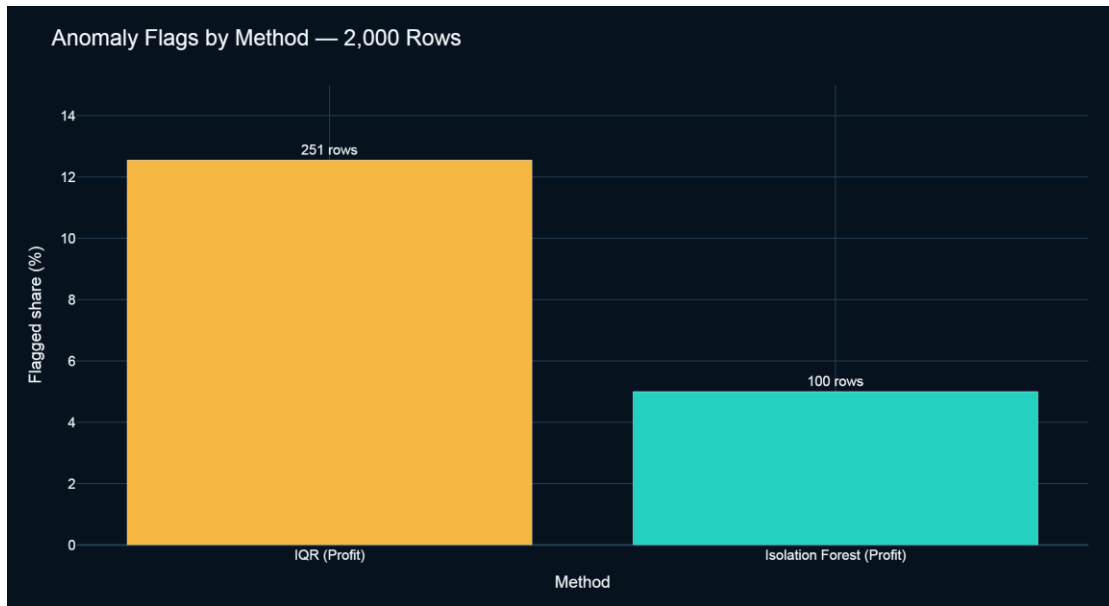


Figure 5. Anomaly rates produced by two complementary methods on Profit.

10.1 Algorithms

The IQR method calculates Q1, Q3, and $IQR = Q3 - Q1$, then flags observations outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$. It is deterministic, transparent, and suited to skewed business measures, but may mark many legitimate tail observations. Isolation Forest constructs random partition trees; observations isolated in fewer splits receive more anomalous scores [8], [9]. The implementation fits 150 trees to the selected numeric target, uses `random_state=42`, and exposes contamination from 0.001 to 0.5. Users may select target, group, and date controls.

Method	Target / setting	Flagged rows	Flagged share
IQR	Profit; $1.5 \times IQR$	251	12.55%
Isolation Forest	Profit; contamination 0.05; 150 trees	100	5.00%

Table 8. Measured anomaly results on the synthetic development dataset.

10.2 Interpretation and evaluation

The methods answer different questions. IQR identifies distributional extremes relative to quartiles; Isolation Forest returns a fixed approximate proportion determined by contamination and ranks unusual values through tree isolation. The higher IQR rate does not imply inferior accuracy, because no ground-truth anomaly labels exist. Precision, recall, ROC curves, and a confusion matrix would therefore be invalid. The interface separates facts—thresholds, scores, counts, rows, and grouped plots—from interpretations that require business validation. Statistical unusualness is not evidence of fraud, error, or causation.

ML evaluation boundary

This is unsupervised, on-demand analysis. The model is fitted to the active subset; there is no reusable trained artefact and no labelled test set. Evaluation reports reproducibility, parameterization, flag counts, and usability—not classification accuracy.

TASK D5

11 Advanced Analytics II — Comparative Analysis

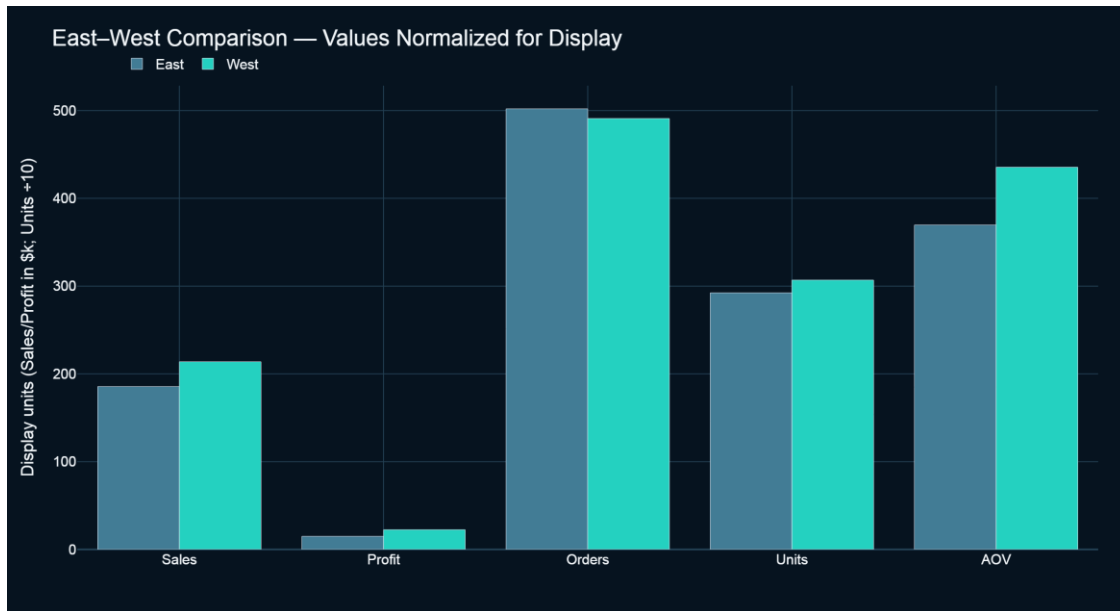


Figure 6. East–West subset comparison; display scaling is stated in the axis title.

11.1 Comparison design

Users define two subsets using Region, Product Category, Customer Segment, Country, or date period. The service calculates Sales, Profit, Orders, Quantity, average order value (AOV), profit margin, average discount, absolute and percentage differences, and row counts. It handles empty subsets, unequal sample sizes, and zero denominators. The chart, detail table, and narrative are generated from the same ComparisonResult contract.

KPI	East	West	West vs East
Sales	\$185,584.52	\$213,841.94	+15.23%
Profit	\$14,924.37	\$22,444.72	+50.39%
Orders	502	491	–2.19%
Units	2,924	3,069	+4.96%
AOV	\$369.69	\$435.52	+17.81%
Profit margin	8.04%	10.50%	+2.45 pp
Average discount	14.35%	12.76%	–1.59 pp
Sample rows	502	491	Comparable sizes

Table 9. Measured East–West comparison on the synthetic development dataset.

11.2 Evidence-based interpretation

West generated 15.23% more sales and 50.39% more profit despite 2.19% fewer orders. Its AOV was 17.81% higher, its margin was 2.45 percentage points higher, and average discount was 1.59 percentage points lower. These are descriptive associations, not causal effects. The similar row counts reduce—without eliminating—sample-size concern; product mix, timing, and customer composition may explain the differences.

ENGINEERING DELIVERY

12 Implementation Process and DevOps

12.1 Technology stack

Layer	Technology	Role
Presentation	Streamlit + CSS	Responsive navigation, state, accessibility, interaction
Data	pandas + PyArrow	Typed transformations and file interoperability
Analytics	DuckDB	In-process parameterized analytical SQL
AI	Google GenAI / OpenAI / Ollama	Provider-neutral structured plan generation
Validation	Pydantic + sqlglot	Typed responses and SQL AST policy
ML	scikit-learn	Isolation Forest anomaly detection
Visualization	Plotly + Kaleido	Interactive charts and PNG/SVG export
Reporting	python-docx + ReportLab	Word and PDF decision artefacts
Quality	pytest, coverage, Bandit, pip-audit	Correctness, coverage, security checks
Operations	Docker, GitHub Actions, Streamlit Cloud docs	Reproducible deployment

Table 10. Principal technologies and responsibilities.

12.2 Repository structure and scalability

The root `app.py` performs configuration and routing. Business logic is organized under `src/data`, `src/llm`, `src/visualization`, `src/advanced`, `src/reporting`, `src/ui`, and `src/utlis`. Tests mirror these concerns. Controlled-path and upload loaders enforce a configurable 200 MB limit; files over one million rows use lazy DuckDB scans to avoid loading the full dataset into pandas. Fresh connections, bounded outputs, modular providers, and stateless report payloads support horizontal Streamlit session scaling, subject to external session and storage infrastructure.

12.3 Version control and delivery

The public repository contains source code, assets, configuration, documentation, tests, benchmark definitions, CI configuration, Docker files, a security policy, contribution guidance, citation metadata, and tagged release v1.4.0. Five meaningful commits document release packaging, preview documentation, interface redesign, hardening, and adaptive AI/branding. No artificial history was generated. External cloud deployment was not claimed because no deployment account was available.

Reproducibility

Target runtime: Python 3.11. The measured workstation used Python 3.12.13, Windows 10 build 19045, AMD64 (4 physical / 8 logical CPUs), and 31.95 GB RAM. Dependencies are pinned and container configuration is provided.

VERIFICATION

13 Testing and Evaluation

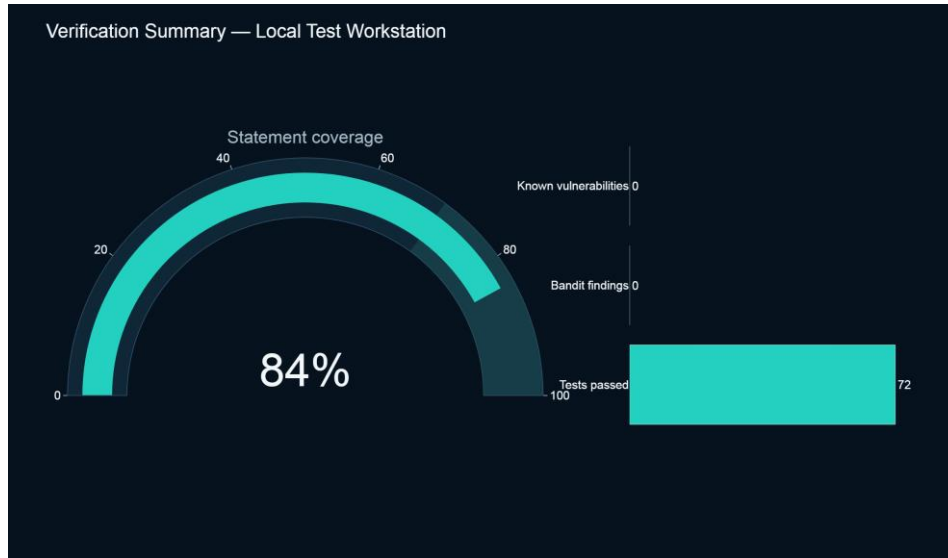


Figure 7. Consolidated correctness, coverage, and security verification summary.

13.1 Test strategy

Test class	Representative coverage
Data	Empty/malformed uploads, aliases, conversion, immutability, profiles, duplicates, quality logs
Queries	Aggregations, simultaneous filters, ranges, sorting, top/bottom N, row limits, empty results
Security	DDL/DML, multi-statements, external scans, imports, dangerous attributes/built-ins, injection cells
Visualization	Typed chart selection plus smoke tests for all eight required constructors
Advanced	IQR, Isolation Forest, subset KPIs, zero division, unequal/empty subsets, missing columns
Exports	Word/PDF generation, document reopening, PDF text, PNG/SVG signatures, missing chart
Application	No-key startup, demo banner, navigation, filter reset, conversation reset, empty states

Table 11. Test coverage by subsystem.

13.2 Measured results

The final local verification run completed 72 tests successfully with 84% statement coverage. Bandit reported zero unsuppressed findings; pip-audit found no known dependency vulnerabilities; the secret scan was clean; and the headless Streamlit health endpoint returned HTTP 200. Export smoke tests produced valid Word, PDF, PNG, and SVG artefacts. Docker runtime build validation was not performed because Docker was not installed on the evaluation workstation, which remains a documented gate.

13.3 Evaluation validity

Automated tests demonstrate conformity to implemented specifications, not business correctness for all possible retailer datasets. AppTest validates framework behavior but does not replace assistive-technology audits or a full cross-browser device laboratory. Security tests reduce known attack surface but cannot prove the absence of vulnerabilities; dependency and provider policies must be reviewed continuously.

MEASURED LATENCY

14 Performance Analysis

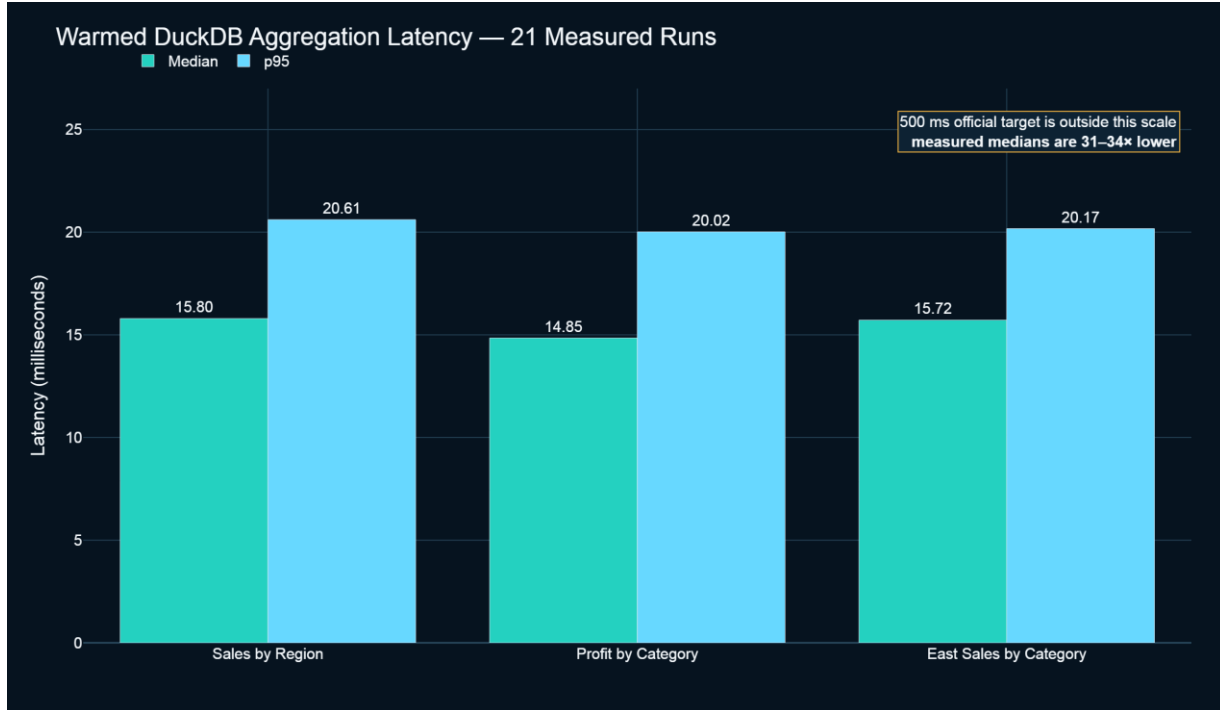


Figure 8. Warmed median and p95 latency for three representative DuckDB aggregations.

Operation	Median	p95	Runs
Sales by Region	15.7956 ms	20.6105 ms	21
Profit by Product Category	14.8455 ms	20.0162 ms	21
East Sales by Product Category	15.7186 ms	20.1737 ms	21

Table 12. Warmed filtered-aggregation benchmark results.

14.1 Loading and profiling

Measured load time was 304.8952 ms and schema profiling took 71.6971 ms for the 2,000-row, 14-column synthetic CSV. The raw pandas frame occupied 0.3924 MB. Query timing used a warmed connection path followed by 21 measured runs for each operation; median and p95 are reported because they are more informative than a single run.

14.2 Interpretation

All three median latencies are substantially below the capstone target of 500 ms. However, this is not an official-dataset pass: the measured file is smaller than the required 5,000+ row demonstration dataset, hardware differs between environments, and hosted-model latency was not measured. The correct conclusion is that the deterministic local analytical path is fast on the stated workstation and dataset. A final submission should rerun the same benchmark after loading the approved official dataset and record its dimensions and hardware.

EVALUATION BOUNDARY

15 AI Benchmark, Reliability, and Result Integrity

15.1 Ten-question benchmark design

ID	Question intent	Expected operation	Chart
1	Highest-sales region	SUM	bar
2	Top five profitable sub-categories	SUM + top 5	bar
3	Monthly sales and profit trend	SUM + time	line
4	Highest segment AOV	AVG	bar
5	East vs West sales	SUM + filter	bar
6	Discount–profit association	correlation	scatter
7	Loss-making countries	SUM + <0	bar
8	Top ten orders by sales	top 10	bar
9	Highest average-profit ship mode	AVG	bar
10	High-discount negative-profit orders	filter	scatter

Table 13. Stored live-provider benchmark specification.

15.2 Scoring and current status

The runner evaluates expected columns, operation and filters, formatting compliance, execution success, chart suitability, and evidence-grounded narrative. Accuracy and completeness scores are meaningful only after an actual provider invocation. At report generation time, all ten rows in `benchmark_results.csv` are `not_run` because no API key was configured; score cells are blank and `execution_success` is false by design. The benchmark command replaces these placeholders with measured results only when explicit live-test configuration is present.

No fabricated AI metrics

The report does not convert deterministic unit-test success into LLM accuracy. Hosted-model accuracy, latency, token usage, and retry rate remain unmeasured. This preserves academic validity and complies with the assignment’s prohibition on fabricated live results.

15.3 Reliability mechanisms

Reliability is improved through schema grounding, provider-native structured outputs, Pydantic validation, AST security checks, parameterized execution, one corrected-query attempt, bounded transport retry, current-filter precedence, five-turn compact memory, evidence-only narratives, numerical caption checks, and deterministic fallback. These mechanisms validate syntax and computation; they do not prove that a user’s question is unambiguous or that the dataset reflects business reality.

CRITICAL REFLECTION

16 Limitations, Ethics, Privacy, and Responsible AI

16.1 Technical and analytical limitations

- The bundled 2,000-row synthetic dataset is suitable for development but not a substitute for the official 5,000+ row demonstration dataset or production data diversity.
- Hosted-model quality and latency vary with provider, model version, quotas, and prompt complexity; the live benchmark is currently not run.
- The deterministic planner covers known e-commerce intents and is intentionally less flexible than a hosted foundation model.
- Isolation Forest is univariate in the current implementation; multivariate context and labelled anomaly validation are future work.
- Choropleth accuracy depends on recognizable geography; the interface falls back to ranked bars when mapping is unreliable.
- Streamlit is appropriate for the capstone scope but enterprise scale requires external authentication, persistent storage, observability, and multi-tenant isolation.
- Automated accessibility and security checks do not replace expert manual audits, user studies, or penetration testing.

16.2 Privacy and governance

Uploaded values are processed locally for deterministic analytics. When a hosted provider is selected, only bounded schema context and truncated samples are included. Users must confirm that data is approved for the selected provider and review current provider terms. Sensitive business data should use an appropriately governed paid endpoint or local Ollama deployment. Secrets are read from environment configuration and are not stored in reports, logs, or conversation records.

16.3 Responsible use

The platform presents AI as an interpretation and planning aid, not an autonomous decision-maker. Generated plans are visible; validation and execution status are disclosed; narratives distinguish computed facts from interpretations; anomalies are explicitly non-causal; and reports include limitations and an AI-validation disclaimer. Users remain responsible for checking field definitions, business context, material decisions, and applicable policy.

16.4 AI assistance disclosure

Generative AI assistance was used to help scaffold, implement, test, review, and document the software and academic artefacts. The project author remains responsible for validating the repository, understanding submitted code and claims, rerunning final benchmarks, replacing placeholders, and complying with institutional authorship and citation rules.

RESEARCH OUTCOME

17 Future Work and Conclusion

17.1 Future improvements

Priority	Improvement	Expected contribution
1	Load the approved 5,000+ row dataset and run the official performance and live-provider benchmark	Completes final empirical gates
2	Add governed authentication, row-level authorization, encrypted object storage, and audit events	Supports multi-user deployment
3	Extend anomaly detection to multivariate and seasonal models with labelled expert review	Improves contextual anomaly quality
4	Add provider telemetry, prompt/version registry, golden datasets, and regression thresholds	Detects AI quality drift
5	Conduct WCAG 2.2 manual audit and task-based usability study across browsers and mobile devices	Strengthens accessibility evidence
6	Introduce semantic metric definitions and dataset contracts	Reduces ambiguity across organizations
7	Deploy to a managed environment with observability, rate limiting, backups, and disaster recovery	Improves operational resilience

Table 14. Prioritized research and engineering roadmap.

17.2 Conclusion

The project demonstrates that natural-language analytics can be made more accessible without surrendering control of computation. Its most important contribution is the explicit boundary between interpretation and execution: a local or hosted model proposes a typed plan, while Pydantic, SQL/pandas AST policies, DuckDB, result summarization, and chart validation determine what is accepted and shown. This design supports flexible interaction while remaining useful without an API key.

Measured local evidence confirms a coherent engineering baseline: real multi-format ingestion, reversible cleaning, six application sections, eight visualization families, two advanced analytical features, exportable reports and charts, 72 passing tests, 84% coverage, clean dependency/security scans, and approximately 15–16 ms warmed query medians on the stated synthetic dataset and workstation. These results should not be generalized beyond the documented conditions. The official dataset, live model benchmark, deployment account, and manual accessibility/security reviews remain transparent final gates.

Final outcome

A professional, reproducible, provider-neutral analytics platform and evidence pack that satisfy the implemented capstone scope while preserving academic honesty about unmeasured external results.

SOURCES

References and Submission Disclosure

- [1] Capstone Project Assignment: AI Analytics, MSc Summer 2026, supplied assessment brief, 2026.
- [2] Streamlit, “Session State,” Streamlit Documentation. https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state (accessed 12 Aug. 2026).
- [3] DuckDB Foundation, “Python API Overview,” DuckDB Documentation. <https://duckdb.org/docs/stable/clients/python/overview> (accessed 12 Aug. 2026).
- [4] pandas Development Team, “pandas 3.0.5 documentation.” <https://pandas.pydata.org/docs/> (accessed 12 Aug. 2026).
- [5] Plotly, “Plotly Python Open Source Graphing Library.” <https://plotly.com/python/> (accessed 12 Aug. 2026).
- [6] T. Gorelik, “sqlglot: Python SQL Parser and Transpiler,” GitHub. <https://github.com/tobymao/sqlglot> (accessed 12 Aug. 2026).
- [7] Google AI for Developers, “Structured outputs — Gemini API.” <https://ai.google.dev/gemini-api/docs/structured-output> (accessed 12 Aug. 2026).
- [8] scikit-learn developers, “IsolationForest,” scikit-learn 1.9.0 documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html> (accessed 12 Aug. 2026).
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in Proc. 8th IEEE ICDM, 2008, pp. 413–422, doi:10.1109/ICDM.2008.17.
- [10] Pydantic Services Inc., “Pydantic documentation.” <https://docs.pydantic.dev/> (accessed 12 Aug. 2026).
- [11] OWASP Foundation, “LLM Prompt Injection Prevention Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html (accessed 12 Aug. 2026).
- [12] OpenAI, “Responses API.” <https://platform.openai.com/docs/api-reference/responses> (accessed 12 Aug. 2026).
- [13] Ollama, “Structured outputs.” <https://docs.ollama.com/capabilities/structured-outputs> (accessed 12 Aug. 2026).
- [14] ReportLab, “ReportLab PDF Toolkit.” <https://www.reportlab.com/docs/reportlab-userguide.pdf> (accessed 12 Aug. 2026).

Submission checklist

- Replace student ID, team-member, submission-date, and deployed-URL placeholders.
- Load the approved 5,000+ row dataset and rerun pytest, security scans, warmed performance benchmark, and 10-question live-provider benchmark.
- Review institutional AI-assistance policy and retain this disclosure or adapt it with supervisor approval.
- Confirm the final PDF remains within 20 pages and the 15-minute presentation timing is rehearsed.

Repository and artefact record

Repository: <https://github.com/Monokayser/ai-ecommerce-analytics>
 Release: <https://github.com/Monokayser/ai-ecommerce-analytics/releases/tag/v1.4.0>
 Live deployment: [INSERT DEPLOYED WEBSITE URL AFTER PUBLICATION]